Implementing Urgent Tasks Function

https://gemini.google.com/app/728afbcd4ebe491f

User prompt: task_manager.py: ```python from datetime import date def add_task(tasks, title, due_date=None, priority="medium"): """Add a new task to the task list. Args: tasks (list): The list of existing task dictionaries. title (str): The task's title. due_date (date, optional): The task's due date. Defaults to None. priority (str): One of "high", "medium", "low". Defaults to "medium". Returns: list: The updated list of tasks. """ tasks.append({"title": title, "due_date": due_date, "priority": priority, "completed": False}) return tasks def mark_complete(tasks, title): """Mark a task as completed by title. Args: tasks (list): The list of task dictionaries. title (str): The title of the task to mark complete. Returns: bool: True if a matching task was found and updated, False otherwise. """ for task in tasks: if task["title"] == title: task["completed"] = True return True return False def get_overdue_tasks(tasks, current_date): overdue = [] for task in tasks: if task["due_date"] < current_date and not task["completed"]: overdue.append(task) return overdue def calculate_completion_rate(tasks): if not tasks: return 0.0 completed = sum(1 for t in tasks if t["completed"]) return completed / len(tasks) ``` Implement get_urgent_tasks(tasks, today) with this explicit specification. Exclude all completed tasks. Include an incomplete task if its priority is high OR it has a due date no later than today + timedelta(days=3); this includes all overdue dates, today, and exactly three days ahead. A non-high task due four days ahead is excluded. A high-priority task is included even when due later or undated. Return a new list sorted by earliest due date first across all priorities, placing undated high-priority tasks last, including after an actual date.max. Preserve input order for equal dates. Do not modify the input list or dictionaries. Return [] for empty input. Assume valid task dictionaries with date or None due dates. Include a Google-style docstring with Args: and Returns:. Return only the necessary standard-library imports and function in one Python code block.

Response:

```
from datetime import timedelta, date

def get_urgent_tasks(tasks, today):
    """Retrieve and sort urgent tasks that are incomplete.

    Args:
        tasks (list): A list of task dictionaries, each containing 'title',
            'due_date', 'priority', and 'completed' keys.
        today (date): The current reference date.

    Returns:
        list: A new list of urgent task dictionaries sorted by due date, with
            undated high-priority tasks placed last.
    """
    if not tasks:
        return []

    threshold = today + timedelta(days=3)
    urgent_tasks = []

    for task in tasks:
        if task.get("completed"):
            continue

        priority = task.get("priority")
        due_date = task.get("due_date")
```

```
        is_high = priority == "high"
        is_due_soon = due_date is not None and due_date <= threshold

        if is_high or is_due_soon:
            urgent_tasks.append(task)

    def sort_key(task):
        due = task.get("due_date")
        if due is None:
            return (1, date.max)
        return (0, due)

    return sorted(urgent_tasks, key=sort_key)
```
